

An Improved Verifiable Database at Page-Level Tampering Detection

Atul Makwana
Dhirubhai Ambani University
DA-IICT, Gandhinagar, India
202411051@daiict.ac.in

Manik Lal Das
Dhirubhai Ambani University
DA-IICT, Gandhinagar, India
maniklal_das@daiict.ac.in

Abstract—Verifiable database helps in identifying data manipulation, particularly when data is stored in a non-quiescent state. The VeriDB scheme [3] proposed a verifiable database mechanism that can detect unauthorized modification of the data. However, the VeriDB scheme detects an integrity breach of the entire data but does not exactly detect the particular location of the data in which the breach occurs. In this paper, we provide an improved verification algorithm that detects data manipulation at the page level. We replace single global ReadSet/WriteSet hashes [3] with per-page hash maps, maintaining page-level ReadSet/WriteSet hash-based aggregates during each Read and Write. A background non-quiescent scan recomputes hashes for each page at a time individually, and compares each page’s last known hash against its freshly computed counterparts, thereby, the proposed algorithm generates an explicit list of tampered page identities. The proposed algorithm is implemented with test cases that show its efficacy in identifying data manipulation and its exact locations.

Index Terms—Verifiable Database, Non-Quiescent Verification, Page-Level Integrity, Trusted Execution Environment, Intel SGX, ReadSet/WriteSet Hashing.

I. INTRODUCTION

Outsourcing storage to the third-party cloud server poses concerns about data integrity and verification of data for usage in the target applications. Generally, cloud services offer benefits like lower costs, scalability, and operational efficiency; however, these benefits come with challenges: the cloud service provider may tamper with the stored data or it may return falsified query results without altering the underlying data. Software Guard Extensions (SGX) based [1] solutions for verifiable databases, such as Concerto [2] leveraged enclave-protected memory to perform offline integrity checks, but the solution was limited to simple key-value stores. VeriDB [3] extended the idea of SGX-based solution to full relational workloads by introducing a non-quiescent verification scheme, which detects the tampering of ReadSets (RS) and WriteSets (WS) [4] without pause in database operations. However, VeriDB’s existing non-quiescent verification operates at a global level, where it maintains two hash aggregates ReadSet and WriteSet that represent the entire address space, and after each scan, it compares these single global hashes and reports whether some inconsistency exists. Although VeriDB’s approach suffices to detect unauthorized modifications, it helps administrators to treat the entire dataset as suspect rather than pin-pointing the exact address space(s). The lack of

localization of tamper detection forces operators into an all-or-nothing recovery strategy: any detected inconsistency forces administrators to treat the entire dataset as suspect. In practice, it requires rolling back or reloading the entire database snapshot, even when only a small fraction of the pages were actually altered.

To address the limitations of the VeriDB’s approach, we propose an improved scheme of non-quiescent verifiable databases, which returns a precise list of address space(s) where integrity checks failed. The background verification scan is modified to re-hash pages individually and then compare each page’s freshly computed WriteSet hash against its stored value, ensuring that these enhancements introduce only minimal enclave memory and CPU overhead and seamlessly integrate with VeriDB’s existing SGX-protected [1] Read/write.

II. IMPROVED PAGE-SPECIFIC VERIFICATION ALGORITHM

In the proposed verification scheme, rather than comparing two hashes [5] that cover the entire database, we keep a pair of hash values for every page – one that reflects all Reads that are performed on that address space, and another that will rebuild fresh during the next audit. By comparing these per-page values, one can instantly know which page(s), if any, have been altered.

$$h_{RS} : \text{Map}(\text{PageID}, \text{HashValue})$$

$$h_{WS} : \text{Map}(\text{PageID}, \text{HashValue})$$

Here, $h()$ indicates a secure hash algorithm (e.g., SHA256 hash function [5]). In Algorithm 1, every memory access, read or write operation, carries out with a built-in integrity check scoped to the specific page being touched. Both operations remain inside the SGX enclave [1], so that the enclave maintains an authoritative and tamper-resistant state. To support non-quiescent verifiability under secure execution, the modified read/write algorithm maintains per-page hash sets to track access operations in a verifiable manner. The system uses two types of hash sets for each page: h_{RS} and h_{WS} for tracking the read and write operations, respectively, during normal execution, and h_{RS}^{new} and h_{WS}^{new} for tracking the read and write operations, respectively, that occur during the verification

process. These hash sets are initialized to zero at the system's startup for each page in the page map.

A. Verifiable Read

For every read operation, a pseudo-random function (PRF) is computed over the combination of the address and the data read. If the address is outside the verification scope (i.e., $\text{addr} > \text{currentScanning}$), the system XORs the PRF result into both h_{RS} and h_{WS} of the corresponding page – this models a virtual write-back after read, ensuring integrity in the absence of quiescence. If a read occurs during verification ($\text{addr} \leq \text{currentScanning}$), the hash updates are instead applied to the h_{RS}^{new} and h_{WS}^{new} sets to avoid polluting the verified state.

B. Verifiable Write

For every write operation, the algorithm first XORs the PRF of the old data (at that address) into the corresponding read hash set – this ensures that prior data access is accounted for. Then, depending on whether the system is in verification mode, it updates the h_{WS} or h_{WS}^{new} hash set with the PRF of the new data. This separation allows the system to concurrently support live database operations and background verification scans, while preserving the cryptographic integrity guarantees. By separating operational and verification-specific hash states, the proposed algorithm enables lightweight non-blocking tamper detection in page-structured storage systems – a foundational principle in systems like VeriDB. By localizing both ReadSet and WriteSet updates to the page identified by each access, the algorithm ensures that each page carries out its own hash of exactly what was read and written. In the background audit, the algorithm simply recomputes and compares the per-page hashes to discover any unauthorized changes and identify which pages require investigation.

The updated non-quiescent verification algorithm enhances the prior model by separating verification-time computations from runtime state, thereby, improving correctness and enabling in-place verification with minimal disruption. The Algorithm 2 introduces per page h_{RS} and h_{WS} hash sets used exclusively during verification. The verifier scans each page in memory sequentially. For each page, it initializes the corresponding h_{WS}^{new} to zero and begins iterating over the page's content as a series of (addr, data) pairs. For each pair, it computes the PRF and XORs it into both h_{RS} and h_{WS}^{new} . Once all entries in a page are processed, the verifier checks if $h_{RS}[\text{pageID}]$ matches the previously stored $h_{WS}[\text{pageID}]$ – the expected hash from runtime operations. A mismatch implies that the page has diverged from what the enclave believes was last written, indicating potential tampering or loss of integrity; in such cases, the `pageID` is added to the `breachedPages` list. After the check, the h_{RS}^{new} and h_{WS}^{new} buffers are rotated: the new values replace the current h_{RS} and h_{WS} , and the temporary buffers are reset to zero for the next page. The buffer rotation ensures the system maintains a consistent verifiable state across scans without blocking or halting regular reads and writes. When all pages have been verified, the

Algorithm 1: Modified Verifiable Read and Write Operation

```

currentScanning = -1
// Initialize sets (globally execute once)
foreach pageID  $\in$  PageMap do
     $h_{RS}[\text{pageID}] \leftarrow 0;$ 
     $h_{WS}[\text{pageID}] \leftarrow 0;$ 
     $h_{RS}^{\text{new}}[\text{pageID}] \leftarrow 0;$ 
     $h_{WS}^{\text{new}}[\text{pageID}] \leftarrow 0;$ 

Function Read (addr, page, data)
    if addr > currentScanning then
         $h_{RS}[\text{page.id}] \leftarrow$ 
             $h_{RS}[\text{page.id}] \oplus \text{PRF}(\text{addr}, \text{data});$ 
         $h_{WS}[\text{page.id}] \leftarrow$ 
             $h_{WS}[\text{page.id}] \oplus \text{PRF}(\text{addr}, \text{data});$ 
    else
         $h_{RS}^{\text{new}}[\text{page.id}] \leftarrow$ 
             $h_{RS}^{\text{new}}[\text{page.id}] \oplus \text{PRF}(\text{addr}, \text{data});$ 
         $h_{WS}^{\text{new}}[\text{page.id}] \leftarrow$ 
             $h_{WS}^{\text{new}}[\text{page.id}] \oplus \text{PRF}(\text{addr}, \text{data});$ 

Function Write (addr, page, dataOld, dataNew)
    if addr > currentScanning then
         $h_{RS}[\text{page.id}] \leftarrow$ 
             $h_{RS}[\text{page.id}] \oplus \text{PRF}(\text{addr}, \text{dataOld});$ 
         $h_{WS}[\text{page.id}] \leftarrow$ 
             $h_{WS}[\text{page.id}] \oplus \text{PRF}(\text{addr}, \text{dataNew});$ 
    else
         $h_{RS}^{\text{new}}[\text{page.id}] \leftarrow$ 
             $h_{RS}^{\text{new}}[\text{page.id}] \oplus \text{PRF}(\text{addr}, \text{dataOld});$ 
         $h_{WS}^{\text{new}}[\text{page.id}] \leftarrow$ 
             $h_{WS}^{\text{new}}[\text{page.id}] \oplus \text{PRF}(\text{addr}, \text{dataNew});$ 

```

algorithm returns a precise list of tampered pages, offering page-level granularity for integrity breach detection.

III. ILLUSTRATION AND RESULTS

Verification of databases involves scanning all memory pages. In the proposed algorithms, normal memory accesses are not blocked during the page verification. Instead, if the application reads/writes a page that has been scanned in the current epoch, the updates go to the new sets; otherwise, they go to the current sets [5]. An epoch is considered complete when the verification of all memory pages finishes.

Case 1: Write only on different pages

Suppose that the scanning of pages begins with a completely clean state: for both pages 1 and 2, the initial ReadSet and WriteSet hashes are zero. The algorithm performs two write operations: first it stores the value “Abc” at address 11 on page 1, and then “Pqr” at address 21 on page 2. Because no prior reads have occurred at those addresses, neither write changes

Algorithm 2: Page-Level Non-Quiescent Verification

```

Function Verify()
  breachedPages  $\leftarrow$  [];
  // Scan every page one by one
  foreach page  $\in$  memory do
    pageID  $\leftarrow$  page.id;
    // Recompute per-page ReadSet
    and WriteSet
    foreach (addr, data)  $\in$  page do
       $h_{RS}[pageID] \oplus = \text{PRF}(addr, data)$ ;
       $h_{WS}^{new}[pageID] \oplus = \text{PRF}(addr, data)$ ;
    end
    // Detect tampering on this page
    if  $h_{RS}[pageID] \neq h_{WS}[pageID]$  then
      breachedPages.append(pageID);
    end
    // Rotate buffers: make 'new'
    the 'current' then clear them
     $h_{RS}[pageID] \leftarrow h_{RS}^{new}[pageID]$ ;
     $h_{WS}[pageID] \leftarrow h_{WS}^{new}[pageID]$ ;
     $h_{RS}^{new}[pageID] \leftarrow 0$ ;
     $h_{WS}^{new}[pageID] \leftarrow 0$ ;
  end
  // List of all pages that failed
  integrity
  return breachedPages;
end

```

Page	1	2
WS	(11, Abc)	(21, Pqr)
RS	0	0
	Write	Write

TABLE I: Page State after Write Operations

the ReadSet hash. Both pages ReadSet remain 0, but each write does update its page's WriteSet so that

- $h_{WS}[1] = \text{PRF}(11, \text{"Abc"})$
- $h_{WS}[2] = \text{PRF}(21, \text{"Pqr"})$

For page-level verification, the verifier zeroes out two new hash maps, h_{RS}^{new} and h_{WS}^{new} , for every page. It then visits each page:

1) **Page 1**

- Scan its stored pair (11, "Abc"). Both

$$h_{RS}[1] \oplus = \text{PRF}(11, \text{"Abc"})$$

$$h_{WS}^{new}[1] \oplus = \text{PRF}(11, \text{"Abc"})$$

- Compare the freshly computed $h_{RS}^{new}[1]$ (which now equals $\text{PRF}(11, \text{"Abc"})$) to the stored $h_{WS}[1]$. They match, so page 1 passes.

2) **Page 2**

- Process (21, "Pqr"), accumulating

$$h_{RS}[2] = \text{PRF}(21, \text{"Pqr"})$$

$$h_{WS}^{new}[2] = \text{PRF}(21, \text{"Pqr"})$$

- compare $h_{RS}^{new}[2]$ against $h_{WS}[2]$. Again, it passes.

No mismatches are found, so `breachedPages` remain empty. Finally, the verifier "rotates" the buffers by copying each page's new hashes into the current maps and resetting the new maps back to zero, so that the system is ready for the next round of reads, writes, or verification. In this way, a simple write to each page produces exactly the expected PRF-based values in both ReadSet and WriteSet, and the verification confirms that the pages are not modified.

Case 2: Write After Write at the Same Address

Page	1	1	2	2
WS	(11, Abc)	(11, Bcd)	(21, Pqr)	(21, Qrs)
RS	0	(11, Abc)	0	(21, Pqr)
	Write	Write	Write	Write

TABLE II: Multiple Write Operations at the Same Address

In this case, writing is done to the *same* address twice on each page, so we can observe how the ReadSet and WriteSet hashes evolve across repeated writes. Initially, both h_{RS} and h_{WS} are zero for all pages.

1. *Writes on Page 1*

- First, write the value "Abc" at address 11. Since there is no previous value at that address, the ReadSet remains unchanged. Therefore, it updates

$$h_{WS}[1] \oplus = \text{PRF}(11, \text{"Abc"})$$

- Then write "Bcd" at the *same* address 11. Before writing, it reads the previous value "Abc" to maintain write-read consistency, which updates

$$h_{RS}[1] \oplus = \text{PRF}(11, \text{"Abc"})$$

and the WriteSet becomes

$$h_{WS}[1] \oplus = \text{PRF}(11, \text{"Bcd"})$$

2. *Writes on Page 2*

- The first write is "Pqr" at address 21. ReadSet stays unchanged, but

$$h_{WS}[2] \oplus = \text{PRF}(21, \text{"Pqr"})$$

- Then it overwrites with "Qrs" at the same address. It first reads "Pqr", so

$$h_{RS}[2] \oplus = \text{PRF}(21, \text{"Pqr"})$$

and write

$$h_{WS}[2] \oplus = \text{PRF}(21, \text{"Qrs"})$$

At this point:

$$h_{RS}[1] = \text{PRF}(11, \text{"Abc"})$$

$$h_{WS}[1] = \text{PRF}(11, \text{"Abc"}) \oplus \text{PRF}(11, \text{"Bcd"})$$

$$h_{RS}[2] = \text{PRF}(21, \text{"Pqr"})$$

$$h_{WS}[2] = \text{PRF}(21, \text{"Pqr"}) \oplus \text{PRF}(21, \text{"Qrs"})$$

Verification Phase

The verifier begins by initializing an empty list to store tampered pages.

It scans both pages as follows:

1. Page 1

- Scans the final state: address 11 with data “Bcd”
- Computes:

$$h_{RS}[1] \oplus = \text{PRF}(11, \text{“Bcd”})$$

$$h_{WS}^{\text{new}}[1] \oplus = \text{PRF}(11, \text{“Bcd”})$$

- Compares $h_{RS}[1]$ to $h_{WS}[1]$, which is:

$$\text{PRF}(11, \text{“Abc”}) \oplus \text{PRF}(11, \text{“Bcd”})$$

- These do matches therefore, page 1 is not added to `breachedPages`.

2. Page 2

- Scans the final value at address 21: “Qrs”
- Computes:

$$h_{RS}[2] \oplus = \text{PRF}(21, \text{“Qrs”})$$

$$h_{WS}^{\text{new}}[2] \oplus = \text{PRF}(21, \text{“Qrs”})$$

- This $h_{RS}[2]$ and $h_{WS}[2]$ also do matches which is:

$$\text{PRF}(21, \text{“Pqr”}) \oplus \text{PRF}(21, \text{“Qrs”})$$

- Therefore, page 2 is not added to `breachedPages`.

Case 3: (Failed Verification Due to Tampering)

Similar to Case 1, two pages that were properly updated through protected write – page 1 holds the value “Abc” at address 11, and page 2 holds “Pqr” at address 21. The enclave’s persistent WriteSet hashes are

$$h_{WS}[1] = \text{PRF}(11, \text{“Abc”})$$

$$h_{WS}[2] = \text{PRF}(21, \text{“Pqr”})$$

and both ReadSet hashes remain zero because no reads have taken place. Now, assume that a malicious actor bypasses our verified write routine and directly patches page 1’s memory, changing the byte at address 11 from “Abc” to “Xyz”. Because this tampering occurs outside the SGX enclave, there is no update to the enclave’s stored hashes. When it triggers a page-level verification, it first zeros out two temporary hash maps h_{RS}^{new} and h_{WS}^{new} for every page. Then it scans each page as follows:

1) Page 1

- Read the single tuple now sitting at (11, “Xyz”). It folds its PRF into both hashes

$$h_{RS}[1] = \text{PRF}(11, \text{“Xyz”})$$

$$h_{WS}^{\text{new}}[1] = \text{PRF}(11, \text{“Xyz”})$$

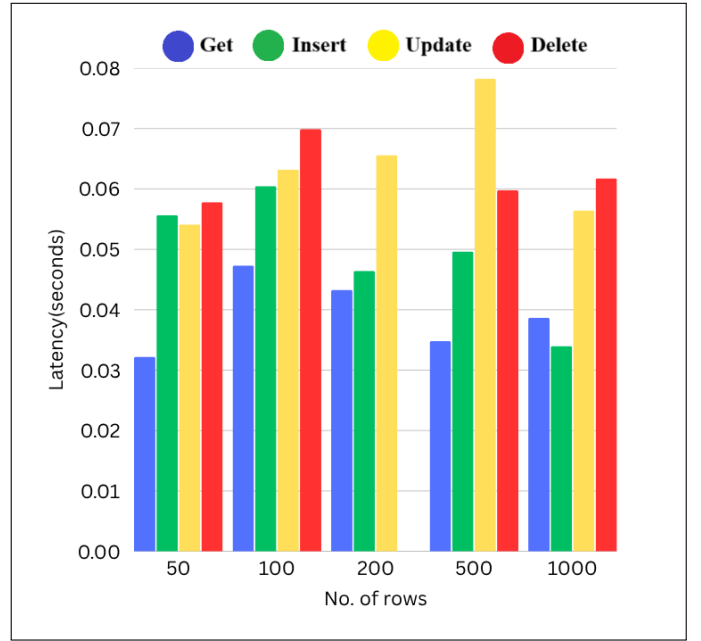


Fig. 1: Performance of Different Operations

- Compare the newly computed ReadSet hash against the enclave’s stored WriteSet hash. Because

$$h_{WS}[1] = \text{PRF}(11, \text{“Abc”})$$

$$h_{RS}[1] \neq h_{WS}[1]$$

- $\Rightarrow$ page 1 fails its integrity check and it records the page identity in `breachedPages`.

2) Page 2

- Compute

$$h_{RS}[2] = h_{WS}[2] = \text{PRF}(21, \text{“Pqr”})$$

- After that it compares the value to the stored $\text{PRF}(21, \text{“Pqr”})$. They match exactly, so page 2 passes.

At the end of the scan, `breachedPages` contains only page identity 1, pin-pointing page 1 as the lone victim of tampering.

IV. EXPERIMENT AND RESULTS

The algorithms are implemented and demonstrated the enhanced non-quiescent verification with a focus on page-level granularity. The implementation simulates a realistic secure storage model in which logical pages track their read and write sets independently. The verification routine is capable of recomputing expected state on a per-page basis and clearly identifies any inconsistencies or potential tampering.

During verification, each page is individually checked, and only those pages whose recomputed hash sets deviate from the maintained state are flagged as `breached`. This localized reporting is crucial in large-scale systems, where isolating the impact of faults or adversarial modifications can reduce recovery cost and diagnostic effort. Additionally, the system

No. of operations	Read	Insert	Update	Delete
50	0.03	0.05	0.05	0.05
100	0.04	0.06	0.06	0.06
200	0.04	0.04	0.06	0.05
500	0.03	0.04	0.07	0.05
1000	0.03	0.03	0.05	0.06

TABLE III: Average Latency of Different Operations (in seconds)

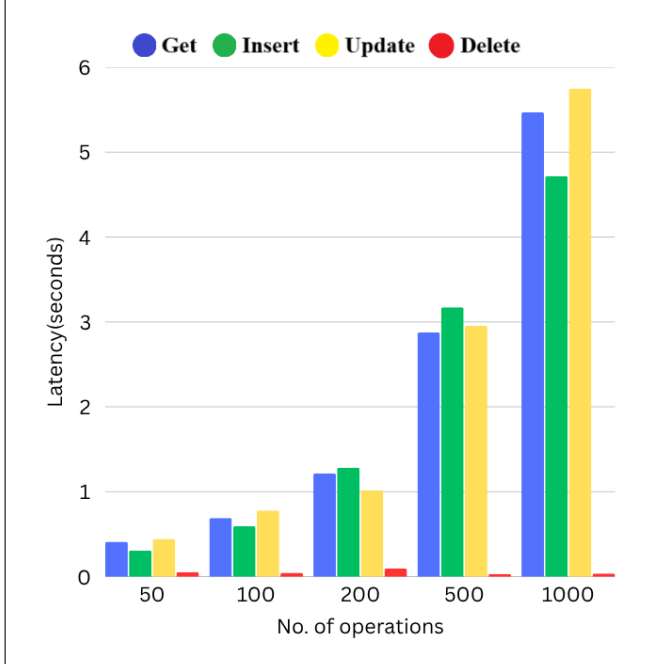


Fig. 2: Performance of Verification Algorithm

supports non-quiescent conditions, meaning verification can proceed concurrently with ongoing operations, which ensures stronger guarantees in dynamic environments, where halting the system for a quiescent snapshot is not always feasible. We evaluated the enhanced page-level non-quiescent verification by measuring both operation latency and verification overhead. Figure 1 shows the average time taken for batches of basic operations (Insert, Update, Delete, and Read) across different dataset sizes. Update and delete operations take slightly longer due to the extra processing involved, while read operations are the fastest. The system maintains consistent performance as the number of operations increases.

TABLE IV: Verification latency (in seconds)

No. of operations	Read	Insert	Update	Delete
50	0.04	0.03	0.04	0.05
100	0.69	0.59	0.77	0.04
200	1.21	1.28	1.01	0.09
500	2.87	3.17	2.95	0.03
1000	5.46	4.71	5.74	0.03

Figure 2 reports the latency of running the verification algorithm after every batch of operations. Verification time grows with the number of operations, especially after 500

actions, as more pages and data need to be checked. Insert and update-heavy workloads result in slightly higher verification costs. In all cases, the algorithm successfully verifies page-level integrity and identifies any modified pages whenever discrepancies are found.

The experiments were carried out on a computer running Ubuntu, equipped with 8 GB of RAM and a 512 GB SSD for storage.

V. CONCLUSION

Verifiable database helps in identifying tampering information in large storage, particularly when data storage is in a non-quiescent state. While the VeriDB scheme [3] can identify an integrity breach of the entire data, we observed that the scheme does not pin-point the exact location of the data where the breach occurs. In this paper, we provided an improved verification algorithm that can detect data manipulation at the page level. We replaced the single global ReadSet/WriteSet hashes with per-page hash maps, maintaining page-level ReadSet/WriteSet hash-based aggregates for each Read and Write. A background non-quiescent scan that recomputes hashes for each page at a time individually. By comparing each page's last known hash against its freshly computed counterpart, the algorithm generates an explicit list of tampered page identities. The proposed algorithm is implemented with test cases that show its efficacy in identifying data manipulation and its exact locations.

REFERENCES

- [1] V. Costan and S. Devadas. Intel SGX Explained. Cryptology ePrint Archive, 2016/086, 2016.
- [2] A. Arasu, K. Eguro, R. Kaushik, D. Kossmann, P. Meng, V. Pandey, and R. Ramamurthy. Concerto: A high concurrency key-value store with integrity. In Proc. ACM Int. Conf. Management of Data (SIGMOD), pp. 251–266, 2017.
- [3] W. Zhou, Y. Cai, Y. Peng, S. Wang, K. Ma, and F. Li. VeriDB: An SGX-based verifiable database. In Proc. Int. Conf. Management of Data (SIGMOD), pp. 2182–2194, 2021.
- [4] M. Blum, W. S. Evans, P. Gemmell, S. Kannan, and M. Naor. Checking the correctness of memories. In Proc. 32nd Annual Symposium Foundations of Computer Science (FOCS), pp. 90–99, 1991.
- [5] M. Sha, Y. Cai, S. Wang, L. T. X. Phan, F. Li, and K.-L. Tan. Object-oriented Unified Encrypted Memory Management for Heterogeneous Memory Architectures. In Proc. ACM Special Interest Group on Data Management, vol. 2, no. 3, Article 155, 2024.
- [6] H. Liu, X. Luo, H. Liu, and X. Xia. Merkle tree: A fundamental component of blockchains. In Proc. Int. Conf. Electronic Information Engineering and Computer Science, pp. 556–561, 2021.
- [7] Y. Zhang, D. Genkin, J. Katz, D. Papadopoulos, and C. Papamathou. vSQL: Verifying arbitrary SQL queries over dynamic outsourced databases. In Proc. IEEE Symposium on Security and Privacy, pp. 863–880, 2017.